

Apostila de Lógica Reconfigurável com VHDL: Fundamentos e Aplicações

Prof. Felipe Walter Dafico Pfrimer

29 de abril de 2026

Sumário

1	Introdução	1
1.1	O que são Dispositivos Lógicos Programáveis?	1
1.2	Linguagens de Descrição de Hardware	2
1.3	O que é VHDL?	2
1.4	Fluxo básico de projeto com VHDL	4
1.4.1	Ferramentas utilizadas na disciplina	5
1.4.2	Próximos capítulos	5
1.5	Perguntas para revisão	5
1.6	Leitura complementar: histórico e evolução da linguagem VHDL	6
1.6.1	Origens e o Programa VHSIC (Década de 1980)	6
1.6.2	Padronização e Evolução (IEEE)	6
1.6.3	Estado da Arte e Aplicação Atual	7
2	Ferramentas e ambiente de desenvolvimento	9
2.1	Visão geral do ambiente	10
2.2	Organização dos arquivos de projeto	10
2.3	Instalação do Visual Studio Code	10
2.4	Configuração da extensão VHDL for Professionals	10
2.5	Instalação do GHDL	10
2.6	Primeira simulação com GHDL	10
2.7	Instalação do VaporView	10
2.8	Visualização de formas de onda	10
2.9	Instalação do Quartus	10
2.10	Criação de um projeto no Quartus	10
2.11	Síntese e implementação no Quartus	10
2.12	Gravação do projeto no FPGA	10
2.13	Automação com Tcl	10
2.14	Fluxo de trabalho recomendado	10
2.15	Problemas comuns e verificação da instalação	10
2.16	Perguntas para revisão	10

Capítulo 1

Introdução

Esta apostila apresenta conceitos fundamentais de VHDL utilizados na disciplina. Este capítulo introduz a base tecnológica dos Dispositivos Lógicos Programáveis (PLDs), a importância das Linguagens de Descrição de Hardware (HDLs) e os primeiros elementos da linguagem VHDL.

1.1 O que são Dispositivos Lógicos Programáveis?

Em um futuro distante, um marceneiro chega ao seu galpão e encontra cubos perfeitos feitos de um material reconfigurável. Com um pequeno dispositivo em mãos, ele carrega um projeto e faz um cubo se transformar em uma mesa; depois, com outro projeto, o mesmo cubo se transforma em uma cadeira ou em uma estante. O objeto físico é o mesmo, mas sua organização interna muda conforme a necessidade.

O segredo está na divisão de tarefas: o dispositivo armazena os *blueprints* (os planos detalhados de cada móvel), enquanto o cubo é feito de um material inteligente, capaz de se moldar dinamicamente, respeitando apenas os limites de sua massa e volume.

Embora soe como ficção científica, essa metáfora ilustra o funcionamento dos **Dispositivos Lógicos Programáveis** (PLDs – *Programmable Logic Devices*): componentes eletrônicos cuja arquitetura interna pode ser configurada pelo usuário para executar funções lógicas específicas, simplesmente carregando um novo “projeto” — ou seja, um novo arquivo de configuração.

No contexto dos PLDs, o “material inteligente” é representado por uma matriz de blocos lógicos e interconexões programáveis. Esses blocos podem ser configurados para realizar operações lógicas variadas, enquanto as interconexões permitem que os blocos se comuniquem, formando circuitos complexos. O *blueprint*, por sua vez, representa o projeto digital. Esse projeto pode ser descrito tanto por diagramas esquemáticos quanto por **Linguagens de Descrição de Hardware** (HDLs).

Ideia-chave

Em um PLD, o hardware não executa uma sequência de instruções como um processador comum. Ele é configurado para se comportar como um circuito digital específico.

Existem diversas categorias de PLDs. Entre as mais comuns estão os **FPGAs** (*Field-Programmable Gate Arrays*) e os **CPLDs** (*Complex Programmable Logic Devices*). Os FPGAs são conhecidos por sua alta densidade lógica e flexibilidade, permitindo a implementação de sistemas digitais complexos. Já os CPLDs costumam ser mais simples, possuem arquitetura menos granular e são ideais para lógicas de controle e aplicações que exigem previsibilidade temporal rigorosa.

1.2 Linguagens de Descrição de Hardware

Uma HDL (*Hardware Description Language*) **não é uma linguagem de programação**. Como o próprio nome sugere, trata-se de uma linguagem utilizada para **descrever** o hardware. Diferentemente das linguagens de software (como C ou Python), que geram instruções sequenciais para um processador, as HDLs permitem que engenheiros definam a estrutura e o comportamento de circuitos eletrônicos, onde muitas coisas acontecem simultaneamente (concorrência).

Antes de continuar

Ao ler exemplos em VHDL, evite procurar uma ordem de execução linha por linha, como em C ou Python. O ponto principal é identificar quais sinais existem, como eles se conectam e que comportamento de hardware está sendo descrito.

Atualmente, as HDLs mais populares para descrever circuitos digitais em PLDs são o **VHDL** (*VHSIC Hardware Description Language*), o **Verilog** e o **SystemVerilog**. Todas permitem a descrição de circuitos em diferentes níveis de abstração, desde o comportamental (o que o circuito faz) até o estrutural (quais portas lógicas compõem o circuito). Esta apostila foca no uso do **VHDL**.

1.3 O que é VHDL?

A sigla VHDL significa **VHSIC Hardware Description Language**, onde VHSIC é o acrônimo para *Very High Speed Integrated Circuit*. Desenvolvida na década de 1980 pelo Departamento de Defesa dos Estados Unidos, a linguagem foi criada para documentar e simular o comportamento de circuitos integrados complexos. Com o tempo, o VHDL evoluiu e se tornou um padrão internacional (IEEE 1076), amplamente adotado na indústria eletrônica para o design de sistemas digitais.

Para exemplificar, considere o Código 1.1, escrito em VHDL, que descreve uma porta AND de duas entradas. Leia este primeiro exemplo como um mapa: os detalhes serão explicados logo abaixo, mas já é possível observar a divisão entre a interface do circuito e seu comportamento interno.

Antes da leitura do código

Antes de tentar entender cada palavra, procure três regiões: as bibliotecas usadas, a entidade que declara entradas e saídas, e a arquitetura que descreve o comportamento da saída.

Código 1.1: Descrição VHDL de uma porta AND de duas entradas

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity AND2 is
5     port (
6         A, B : in  std_logic;
7         Y    : out std_logic
8     );
9 end AND2;
10
11 architecture rtl of AND2 is
12 begin
13     Y <= A and B;
```

```
14 end rtl;
```

Depois da leitura do código

A entidade diz como o circuito aparece para o mundo externo. A arquitetura diz o que acontece dentro dele. Essa separação aparece em praticamente todos os projetos VHDL.

Observe no Código 1.1 a estrutura básica de um arquivo VHDL:

- **Bibliotecas:** As linhas iniciais importam a biblioteca `ieee`, necessária para usar o tipo de dado padrão `std_logic`. Outras bibliotecas podem ser incluídas conforme a complexidade do projeto.
- **Entidade (Entity):** Define a interface do componente, ou seja, suas entradas (A, B) e saídas (Y). Funciona como a “caixa preta” do componente vista de fora. Aqui, o projetista deve prever quais sinais o componente receberá e quais ele fornecerá.
- **Arquitetura (Architecture):** Descreve o comportamento interno. Neste caso, a saída Y recebe o resultado da operação lógica AND entre A e B. É nessa região que os iniciantes em VHDL possuem maior dificuldade, pois já estão acostumados a pensar em termos de sequências de comandos comuns nas linguagens de programação. Dessa forma, o VHDL exige uma mentalidade de descrição concorrente. Na arquitetura, todas as operações são consideradas simultâneas, refletindo a natureza paralela do *hardware* alvo.

Além da descrição para síntese (criação do circuito físico), o VHDL é amplamente utilizado para **simulação**. Isso permite que os desenvolvedores validem o comportamento do circuito no computador antes de implementá-lo no PLD, economizando tempo e garantindo a confiabilidade do projeto.

O código a seguir apresenta um exemplo simples de testbench em VHDL, utilizado para simular o comportamento da porta AND definida no Código 1.1. Ele contém mais elementos que o exemplo anterior; neste momento, o mais importante é perceber que o testbench cria sinais, conecta a porta AND e aplica combinações de entrada ao longo do tempo.

Ao ler o testbench

Não é necessário memorizar a estrutura completa agora. Observe apenas que os comandos `wait for 10 ns` criam intervalos de simulação e permitem verificar como a saída responde a cada combinação de A e B.

Código 1.2: Testbench para validar a porta AND

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity tb_AND2 is
6 end tb_AND2;
7
8 architecture behavior of tb_AND2 is
9     signal A, B : std_logic := '0';
10    signal Y      : std_logic;
11
12    component AND2
```

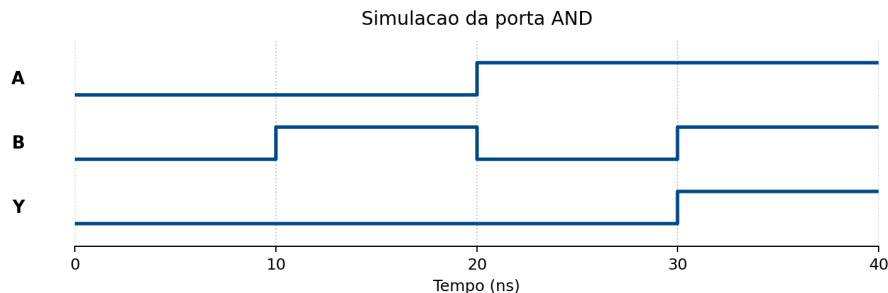
```

13     port (
14         A, B : in  std_logic;
15         Y    : out std_logic
16     );
17     end component;
18 begin
19     uut: AND2 port map (A => A, B => B, Y => Y);
20
21     process
22     begin
23         A <= '0'; B <= '0'; wait for 10 ns;
24         A <= '0'; B <= '1'; wait for 10 ns;
25         A <= '1'; B <= '0'; wait for 10 ns;
26         A <= '1'; B <= '1'; wait for 10 ns;
27     wait;
28     end process;
29 end behavior;

```

No Código 1.2, o testbench cria sinais de entrada (A e B) e conecta a unidade sob teste (UUT - *Unit Under Test*), que é a porta AND definida anteriormente. O processo dentro do testbench aplica diferentes combinações de entradas, aguardando 10 nanosegundos entre cada mudança, permitindo observar a saída Y em resposta às entradas. O resultado da simulação pode ser visualizado em uma forma de onda, facilitando a verificação do comportamento correto do circuito, como pode ser visto na Figura 1.1.

Figura 1.1: Forma de onda da simulação da porta AND



Dessa forma, o VHDL possui duas formas principais de uso:

- **Síntese:** na qual o código VHDL infere o circuito lógico que será implementado no PLD.
- **Simulação:** na qual o código VHDL é utilizado para validar o comportamento do circuito antes da síntese.

Essas duas vertentes são fundamentais no fluxo de projeto digital, garantindo que o design atenda aos requisitos funcionais e de desempenho antes da implementação física. No entanto, cada uma dessas vertentes possui suas próprias regras e boas práticas de codificação, que serão abordadas nos capítulos seguintes. Isso acontece porque certos construtos do VHDL são adequados para simulação, mas não para síntese, e vice-versa. Por exemplo, estruturas como `wait` e `after` são úteis para simulação, mas não podem ser sintetizadas em hardware.

1.4 Fluxo básico de projeto com VHDL

Em um projeto digital com VHDL, o código escrito pelo projetista passa por uma sequência de etapas antes de chegar ao hardware. Em uma visão simplificada, esse fluxo pode ser entendido

da seguinte forma:

Descrição em VHDL → Simulação → Síntese → Implementação no PLD → Teste em hardware

Na etapa de **descrição**, o projetista escreve os arquivos VHDL que representam o circuito desejado. Em seguida, na **simulação**, um testbench é usado para verificar se o comportamento descrito está correto antes de envolver o hardware físico. Se a simulação indicar que o circuito se comporta como esperado, o projeto pode seguir para a **síntese**, etapa em que a descrição VHDL é convertida em uma rede de elementos lógicos.

Depois da síntese, a ferramenta de desenvolvimento realiza a **implementação**, posicionando e conectando esses elementos dentro do PLD escolhido. Por fim, o projeto é carregado no dispositivo e testado em hardware real. Na prática, esse fluxo costuma ser iterativo: erros encontrados na simulação, na síntese ou no teste em hardware levam o projetista de volta ao código VHDL.

Ideia-chave

Simular antes de sintetizar reduz o tempo gasto procurando erros no hardware. Sempre que possível, o comportamento do circuito deve ser validado no computador antes da implementação física.

1.4.1 Ferramentas utilizadas na disciplina

Ao longo da disciplina, serão utilizadas ferramentas para edição, simulação, visualização de formas de onda e implementação em FPGA. A organização prevista é:

- **Visual Studio Code:** ambiente de edição dos arquivos VHDL.
- **GHDL:** ferramenta para compilação e simulação dos projetos VHDL.
- **VaporView:** ferramenta para visualização de formas de onda geradas nas simulações.
- **VHDL for Professionals:** extensão de apoio à edição e análise de código VHDL.
- **Quartus:** ferramenta para síntese, implementação e gravação dos projetos em FPGA.
- **Tcl:** possível apoio para automatizar comandos e fluxos dentro do Quartus.

1.4.2 Próximos capítulos

Os próximos capítulos irão aprofundar gradualmente os elementos apresentados nesta introdução. Esta seção será completada conforme a estrutura final da apostila for definida.

1.5 Perguntas para revisão

1. O que significa dizer que um PLD pode ser configurado pelo usuário?
2. Na metáfora apresentada, o que representa o “material inteligente”? E o que representa o *blueprint*?
3. Por que uma HDL não deve ser entendida como uma linguagem de programação comum?
4. Quais HDLs são citadas como populares para descrever circuitos digitais em PLDs?

5. Qual é a função da entidade em um código VHDL?
6. Qual é a função da arquitetura em um código VHDL?
7. No exemplo da porta AND, quais são as entradas e qual é a saída?
8. Para que serve um testbench?
9. Qual é a diferença entre síntese e simulação?
10. Por que estruturas como `wait` e `after` são úteis para simulação, mas não para síntese?

1.6 Leitura complementar: histórico e evolução da linguagem VHDL

Esta seção apresenta um breve contexto histórico sobre a origem do VHDL. Ela não é necessária para compreender os exemplos anteriores, mas ajuda a entender por que a linguagem possui características como tipagem forte, sintaxe rigorosa e foco em documentação de hardware.

A linguagem VHDL (*VHSIC Hardware Description Language*) possui uma origem distinta da maioria das linguagens de programação de software, tendo sido concebida inicialmente não para a execução, mas para a documentação e preservação de projetos eletrônicos complexos [1].

1.6.1 Origens e o Programa VHSIC (Década de 1980)

No início da década de 1980, o Departamento de Defesa dos Estados Unidos (DoD) enfrentava uma crise logística relacionada ao ciclo de vida de seus componentes eletrônicos. Com o avanço rápido da tecnologia, chips utilizados em equipamentos militares tornavam-se obsoletos rapidamente, e a falta de documentação padronizada impedia a reprodução desses componentes por novos fabricantes [7, 3].

Para solucionar este problema, o DoD iniciou o programa VHSIC (*Very High Speed Integrated Circuits*). O objetivo era criar uma linguagem agnóstica de tecnologia que pudesse descrever o comportamento e a estrutura de circuitos integrados. Em 1983, um contrato foi firmado com a Intermetrics, IBM e Texas Instruments para desenvolver a primeira versão da linguagem. Uma decisão crucial de design foi basear a sintaxe na linguagem Ada, herdando desta a tipagem forte e a não-sensibilidade a letras maiúsculas/minúsculas, visando reduzir erros de ambiguidade em projetos críticos [6].

1.6.2 Padronização e Evolução (IEEE)

Embora criada para documentação, os engenheiros perceberam rapidamente que uma descrição comportamental precisa poderia ser executada por softwares de simulação. Para fomentar a adoção pela indústria civil e garantir a interoperabilidade entre ferramentas de CAD (*Computer-Aided Design*), o DoD transferiu os direitos da linguagem para o IEEE (*Institute of Electrical and Electronics Engineers*) em 1986.

O primeiro padrão oficial foi publicado como **IEEE Standard 1076-1987** (conhecido como VHDL-87). Desde então, a linguagem passou por revisões periódicas para incorporar novas capacidades de síntese e verificação [5]:

- **VHDL-93:** Introduziu uma sintaxe mais consistente, identificadores estendidos e melhorias na manipulação de arquivos.

- **IEEE 1164:** Embora não seja uma revisão da linguagem em si, a padronização do pacote `std_logic_1164` foi um marco complementar fundamental. Ela introduziu o sistema lógico de 9 valores (incluindo 'Z' para alta impedância e 'X' para desconhecido), permitindo a modelagem realista de circuitos digitais.
- **VHDL-2000 e 2002:** Pequenas revisões que adicionaram o modificador `protected` para tipos.
- **VHDL-2008 (IEEE 1076-2008):** Uma grande modernização que incorporou recursos inspirados em SystemVerilog, facilitando a verificação e reduzindo a verbosidade do código para operações comuns.
- **VHDL-2019:** Uma revisão recente, focada em melhorias para interfaces de verificação e introspecção de tipos.

1.6.3 Estado da Arte e Aplicação Atual

Atualmente, o VHDL é uma das linguagens dominantes no design de hardware digital, dividindo o mercado global com o Verilog/SystemVerilog. Seu uso principal migrou da documentação para a **síntese lógica** e a **simulação**.

O VHDL é amplamente empregado no desenvolvimento de FPGAs (*Field-Programmable Gate Arrays*) e ASICs (*Application-Specific Integrated Circuits*). Devido à sua natureza fortemente tipada e determinística, o VHDL mantém uma forte preferência em setores que exigem alta confiabilidade e segurança crítica, como as indústrias aeroespacial, automotiva, médica e de defesa, especialmente na Europa e nas Américas. Ferramentas modernas de síntese da Intel (Quartus) e AMD/Xilinx (Vivado) oferecem suporte abrangente aos padrões mais recentes, permitindo que a linguagem descreva desde portas lógicas simples até processadores *soft-core* complexos e aceleradores de inteligência artificial [2, 4].

Capítulo 2

Ferramentas e ambiente de desenvolvimento

- 2.1 Visão geral do ambiente
- 2.2 Organização dos arquivos de projeto
- 2.3 Instalação do Visual Studio Code
- 2.4 Configuração da extensão VHDL for Professionals
- 2.5 Instalação do GHDL
- 2.6 Primeira simulação com GHDL
- 2.7 Instalação do VaporView
- 2.8 Visualização de formas de onda
- 2.9 Instalação do Quartus
- 2.10 Criação de um projeto no Quartus
- 2.11 Síntese e implementação no Quartus
- 2.12 Gravação do projeto no FPGA
- 2.13 Automação com Tcl
- 2.14 Fluxo de trabalho recomendado
- 2.15 Problemas comuns e verificação da instalação
- 2.16 Perguntas para revisão

Referências Bibliográficas

- [1] Peter J. Ashenden. *The Designer's Guide to VHDL*. Morgan Kaufmann, 3 edition, 2008.
- [2] Pong P. Chu. *FPGA Prototyping by VHDL Examples*. Wiley-Interscience, 2008.
- [3] Allen Dewey. VHSIC hardware description language (VHDL) development program. In *Proceedings of the 20th Design Automation Conference*, 1983.
- [4] Doulos. VHDL history and evolution. Technical article.
- [5] IEEE Computer Society. *IEEE Standard VHDL Language Reference Manual*. IEEE, 2009. IEEE Std 1076-2008.
- [6] IEEE Solid-State Circuits Society. The roots of VHDL. *IEEE Solid-State Circuits Magazine*, 2018.
- [7] United States Department of Defense. Requirements for hardware description languages. Technical report, United States Department of Defense, 1981.